

Deep Learning for Natural Language Processes Report

Abstract

This work evaluates four sentiment classification models of increasing architectural complexity on binary sentiment classification of IMDb movie reviews under constrained experimental conditions of 4,000 training examples and CPU-only compute. The models compared are TF-IDF with logistic regression, Word2Vec with mean pooling and an MLP classifier, an LSTM trained from scratch, and fine-tuned DistilBERT. All models are assessed on accuracy, F1, AUC, and training time to evaluate both predictive performance and computational cost. TF-IDF logistic regression achieves the highest test accuracy (0.8640), F1 (0.8528), and AUC (0.9457), outperforming all three neural models despite training in 1.17 seconds. Fine-tuned DistilBERT achieves the second-highest accuracy (0.8320) but requires over 3,200 seconds of training time, more than three orders of magnitude greater than the logistic regression baseline. The results demonstrate that increased architectural complexity does not reliably produce better predictive performance under limited data and compute constraints, and that task characteristics, specifically the lexical nature of sentiment classification, strongly moderate the benefit of more expressive representations.

1 Introduction

Sentiment analysis, the automatic classification of text according to expressed emotional polarity, is among the most widely studied tasks in natural language processing. Organisations apply sentiment models to product reviews, social media posts, and customer feedback at a scale that makes manual annotation impractical, and the task has become a standard benchmark for evaluating NLP representations because the discriminative signal is well-defined and largely lexical in nature (1).

This report evaluates four modelling approaches on binary sentiment classification of IMDb movie reviews. TF-IDF with logistic regression, Word2Vec with mean pooling and an MLP classifier, an LSTM trained from scratch, and fine-tuned DistilBERT. These four models represent successive generations of NLP methodology, from statistical feature engineering through dense word representations and explicit sequence modelling to pre-trained contextual encoding, and together they allow a systematic comparison of representational assumptions under controlled conditions.

The central question is not simply which model achieves the highest accuracy, but whether increased architectural complexity produces performance gains that are proportional to the associated increases in data requirement and computational cost. All four models are trained on 4,000 labelled examples in a CPU-only environment, conditions that are representative of many real-world deployment settings and that place particular pressure on models

whose advantage depends on large-scale pre-training or data-hungry optimisation. This experimental design makes it possible to isolate the effect of architectural choice from the effect of compute and data scale.

2 Literature Survey

Early work on sentiment classification demonstrated that representing documents as bags of words, combining term frequency features with a linear classifier, was sufficient to achieve competitive accuracy on movie review data (1). The key finding was that opinion text contains strong lexical signals, and that knowing which words appear in a review carries substantial predictive information independently of word order. The principal limitation of this approach is that it assigns no meaning to words beyond their weighted frequency and provides no mechanism for capturing semantic similarity between related terms.

Word2Vec addressed this by training a shallow neural network on a skip-gram objective, producing dense vector representations in which semantically related words are geometrically proximate in the embedding space (2). A critical property of these representations is that their quality depends heavily on the volume of training data, as reliable estimation of co-occurrence statistics requires corpora substantially larger than a typical task-specific dataset. This data dependency is a direct constraint in the present work, where embeddings are trained on the task corpus alone. Both approaches discard word order entirely. The LSTM addressed this by processing text sequentially

and maintaining a learned memory of prior context through three gating mechanisms that control what information is retained, written, and exposed at each time step (3). This resolved the vanishing gradient problem of earlier recurrent architectures and enabled modelling of compositional phenomena such as negation and qualification, but at the cost of requiring sufficient data to learn embeddings and sequential dynamics simultaneously.

The transformer architecture replaced recurrence with self-attention, allowing every token to attend directly to every other token regardless of sequential distance (4). BERT demonstrated that pre-training a transformer encoder on large unlabelled corpora through masked language modelling produces representations that transfer effectively to downstream tasks with minimal fine-tuning (5). DistilBERT applied knowledge distillation to produce a compressed version retaining approximately 97% of BERT’s performance at 40% fewer parameters, making transformer-based classification more feasible under resource constraints (6).

Each successive approach addressed a genuine limitation of the one before it, but also introduced greater demands on data and compute. This suggests that the relationship between model complexity and performance is not unconditional, and depends on the constraints of the experimental setting.

3 Description of models

The four models evaluated in this work represent successive generations of NLP methodology, each designed to address a specific limitation of its predecessor. The progression moves from statistical feature engineering, which requires no learned representations, through dense word embeddings trained on task data, to explicit sequential modelling, and finally to pre-trained contextual encoding. Each step increases representational expressiveness but also introduces greater demands on training data, parameter initialisation, and computational resource. A critical question motivating this comparison is whether these increased demands are justified under the experimental constraints of this work, specifically 4,000 training examples and a CPU-only environment. Figure 1 summarises the end-to-end pipeline for all four models, illustrating how the input representation changes at each stage of the progression before each model is described in detail.

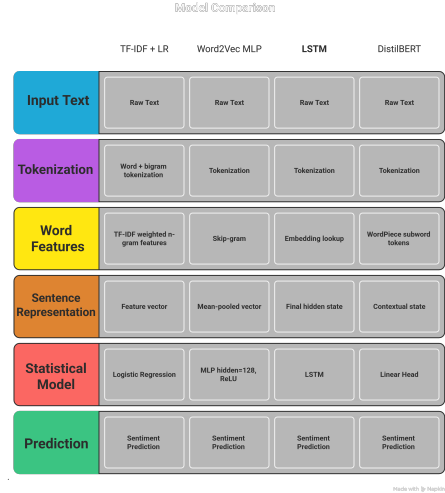


Figure 1: Comparative NLP pipeline across all four models, organised by shared conceptual stages. Each column represents one model and each row represents a fixed pipeline stage, from raw text input through tokenisation, word-level features, sentence representation, and statistical or neural model, to the final sentiment prediction. The word-level features and sentence representation rows highlight where the four models differ most fundamentally in their representational assumptions.

3.1 TF-IDF with Logistic Regression

TF-IDF with logistic regression belongs to the statistical NLP tradition that treats documents as unordered collections of weighted term occurrences. It requires no learned word representations and instead constructs a fixed, interpretable feature space from the training vocabulary. Each review is converted into a sparse vector $\mathbf{x} \in \mathbb{R}^{10000}$, where each dimension corresponds to a word or bigram feature weighted by

$$tf - idf(w, d) = tf(w, d) * \log \frac{N}{df(w)} \quad (1)$$

where $tf(w, d)$ is the frequency of w in d , N is the total number of documents, and $df(w)$ is the number of documents containing w . This weighting rewards terms that appear frequently in a specific document but rarely across the corpus as a whole, directly capturing vocabulary that is locally informative while suppressing common uninformative terms such as function words. Sublinear TF scaling is applied to reduce the influence of very high-frequency terms, preventing a small number of dominant words from overwhelming the feature space. Both unigrams and bigrams are included in the vocabulary, allowing two-word phrases such as "not good" and "highly rec-

ommended" to be treated as single features. This partially addresses the negation problem inherent in purely unigram representations, where the sentiment contribution of a negation word and the word it modifies cannot be jointly captured. The resulting sparse document vector is passed to a logistic regression classifier, which learns a scalar weight for each of the 10,000 vocabulary features and produces a binary prediction via

$$\hat{y} = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (2)$$

Training minimises binary cross-entropy with L2 regularisation, which penalises large weights and prevents overfitting on the high-dimensional sparse feature space. The LBFGS optimiser is selected because it is a quasi-Newton second-order method that approximates the inverse Hessian of the objective function, exploiting curvature information to take more efficient steps than first-order gradient descent. For a convex objective such as regularised logistic regression, LBFGS is guaranteed to converge to the global minimum and does so reliably without requiring careful learning rate tuning. The model learns one scalar weight per vocabulary feature and a single bias term, making predictions directly interpretable: strongly positive weights identify features strongly associated with positive sentiment and strongly negative weights identify features associated with negative sentiment.

The fundamental limitation of this approach is the complete absence of word order, syntactic structure, and semantic relationships between terms. Synonymous words are treated as entirely independent features, and the model cannot represent how words modify one another across a sequence. However, as will be shown in Section 5, this limitation is largely inconsequential for sentiment classification on movie reviews, where the discriminative signal is primarily lexical and where individual words carry strong and consistent sentiment information regardless of their sequential context.

3.2 Word2Vec with Mean Pooling and MLP

Word2Vec addressed the principal representational limitation of bag-of-words methods by replacing sparse frequency features with dense vectors that encode semantic similarity geometrically (2). Rather than treating each word as an independent dimension, the skip-gram objective trains a shallow two-layer neural network to predict surrounding context words given a target word w with

context words c in a window of size k , the objective maximises

$$\frac{1}{|T|} \sum_{w \in T} \sum_{-k \leq j \leq k, j \neq 0} \log P(w_{t+j} | w_t) \quad (3)$$

The result is a set of embedding vectors $e_w \in \mathbb{R}^{50}$ in which semantically related words are geometrically proximate, capturing distributional similarity that TF-IDF cannot represent. In this work, embeddings are trained from scratch exclusively on the 4,000 training reviews using a context window of five. A null token is added to the vocabulary with an embedding equal to the mean of all trained vectors, used to handle both out-of-vocabulary words and padding positions at inference time.

To produce a fixed-size document representation from a variable-length sequence of token embeddings, all token embeddings in a review are averaged through mean pooling

$$v_d = \frac{1}{|T|} \sum_{w \in T} e_w \in \mathbb{R}^{50} \quad (4)$$

where T is the set of tokens in the review. This pooled vector $v_d \in \mathbb{R}^{50}$ is passed through a two-layer MLP with a single hidden layer of dimension 128, ReLU activation, and a linear output layer producing two class logits. The embedding layer is fixed after Word2Vec training and only the MLP weights are updated during classification training. Jointly updating the full embedding table from 4,000 classification examples would risk severely overfitting the already noisy embedding geometry, since the skip-gram objective was trained on the same limited corpus.

This model carries two compounding limitations in this experimental setting. First, mean pooling discards word order entirely. The document representation of "the film was not good" is identical to "the film was good," since the pooling operation is commutative over the token set. The model therefore inherits the sequential blindness of TF-IDF while also losing TF-IDF's principled frequency weighting. Second, and more fundamentally, the skip-gram objective requires large corpora to produce reliable embeddings. Reliable estimation of co-occurrence statistics requires many observations of each word in diverse contexts, and 4,000 reviews is orders of magnitude smaller than the training data typically used for Word2Vec. The resulting embeddings are likely to be noisy and geometrically unreliable for rare or domain-specific sentiment vocabulary, providing a weak foun-

dation for the MLP classifier regardless of its capacity.

3.3 LSTM

The LSTM was designed to process text as an ordered sequence and maintain a learned memory of prior context, directly addressing the word-order blindness shared by both TF-IDF and mean pooling (Hochreiter and Schmidhuber, 1997). Each review is tokenised using a regular expression that strips punctuation from token boundaries, encoded as a sequence of integer indices from a vocabulary of up to 20,000 tokens, and truncated or padded to a maximum length of 200 tokens. An embedding layer maps each index to a learned vector $e_{x_t} \in \mathbb{R}^{100}$, producing the token embedding sequence that is fed into the recurrent network. At each time step t , the LSTM updates its hidden state $h_t \in \mathbb{R}^{128}$ and cell state $c_t \in \mathbb{R}^{128}$ through three learned gating mechanisms that jointly control information flow through the network

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, e_{x_t}] + b_f) \\ i_t &= \sigma(W_i[h_{t-1}, e_{x_t}] + b_i) \\ o_t &= \sigma(W_o[h_{t-1}, e_{x_t}] + b_o) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, e_{x_t}] + b_c) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

The forget gate f_t produces values in (0,1) for each dimension of the cell state, controlling what proportion of the previous cell state c_{t-1} is retained. A forget gate value near zero discards the corresponding dimension of the cell state, while a value near one preserves it. The input gate i_t controls what new information from the current token embedding e_{x_t} is written into the cell state, modulated by a candidate cell state computed through a $\tanh$ nonlinearity. The output gate o_t controls what portion of the current cell state is exposed as the hidden state h_t at each time step, which is passed both to the next time step and, at the final real token position, to the linear classification head. The cell state c_t acts as a long-range memory channel that can in principle propagate relevant sentiment signal across hundreds of time steps with minimal decay, resolving the vanishing gradient problem that made earlier recurrent architectures untrainable on long sequences.

The hidden state at the final real token position, identified using the true sequence length before padding to avoid incorporating padding embeddings into the classification decision, is passed to a linear layer that produces the binary sentiment prediction. All

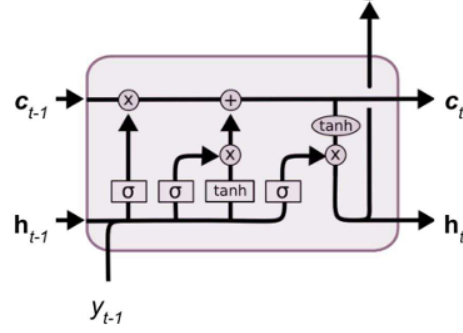


Figure 2: LSTM cell architecture. The forget, input, and output gates regulate updates to the cell state c_t and hidden state h_t , enabling the model to capture sequential dependencies and retain relevant context over time.

parameters, including the embedding layer, recurrent weight matrices, gating matrices, and classifier weights, are randomly initialised and trained jointly from the task data alone. The total parameter count is substantially larger than the Word2Vec MLP, and the optimisation problem is considerably more demanding: unlike the Word2Vec MLP where the embedding layer is fixed and only the MLP weights are updated, the LSTM must simultaneously learn what words mean, how to represent sequences, and how to gate information flow, all from the same 4,000 training examples with no prior knowledge of language. This joint optimisation from random initialisation under data constraints is the central tension in this model’s design and the primary explanation for its results in Section 5.

3.4 DistilBERT

DistilBERT is a transformer-based encoder model produced through knowledge distillation from BERT, retaining approximately 97% of BERT’s performance at 40% fewer parameters (6). It processes text using multi-head self-attention across six transformer encoder layers, in which every token simultaneously attends to every other token in the input through the scaled dot-product attention mechanism (year?)

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

where Q , K and V are learned projections of the input and d_k is the key dimension. The dot product QK^T computes a similarity score between every pair of tokens in the input, measuring how much each token should attend to every other. Scaling by $1/\sqrt{d_k}$ prevents these dot products from growing

large in magnitude as the embedding dimension increases, which would push the softmax into regions of very small gradient and destabilise training. The softmax normalises these scores into a probability distribution over all input positions, and the weighted sum over value vectors V produces a contextualised representation for each token that incorporates information from the full input sequence. This mechanism allows the model to directly represent relationships between any two tokens regardless of their sequential distance, removing the bottleneck of recurrence entirely and enabling richer contextual representations than sequential processing can produce.

Input text is tokenised using a WordPiece subword tokeniser, which decomposes unknown words into known subword units rather than mapping them to a single unknown token, giving the model better coverage of rare and morphologically complex vocabulary than a fixed word-level vocabulary can provide. Sequences are truncated to 128 subword tokens to reduce CPU inference time. A special [CLS] token is prepended to each sequence before tokenisation. After passing through all six encoder layers, the final hidden state of the [CLS] token $h_{CLS} \in \mathbb{R}^{768}$ serves as a fixed-size aggregate representation of the entire input sequence, which is passed to a linear classification head

$$\hat{y} = \text{softmax}(\mathbf{W}h_{CLS} + \mathbf{b}) \quad (6)$$

where $\mathbf{W} \in \mathbb{R}^{2 \times 768}$ maps the 768-dimensional CLS representation to two class logits.

Unlike all three preceding models, DistilBERT enters task-specific training with weights derived from masked language modelling pre-training on approximately 16GB of text from BookCorpus and English Wikipedia. During pre-training, the model learns to predict randomly masked tokens from their bidirectional context, acquiring rich representations of syntax, semantics, and world knowledge that transfer effectively to downstream tasks. Fine-tuning updates both the encoder weights and the classification head using a learning rate of 2×10^{-5} , which is deliberately small to adjust the pre-trained representations toward the task without overwriting the linguistic knowledge encoded in them through large gradient updates. This means DistilBERT effectively has access to linguistic information derived from a corpus several orders of magnitude larger than the 4,000 training examples available to the other three models, giving it a representational advantage that cannot be replicated through task-specific training alone.

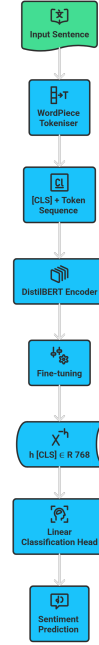


Figure 3: DistilBERT sequence classification architecture. WordPiece-tokenised input, including a prepended [CLS] token, is processed by the pre-trained DistilBERT encoder to produce contextual representations. The final [CLS] hidden state is extracted and passed to a linear head for sentiment prediction.

It is important to be analytically precise about what the comparison between DistilBERT and the other models actually tests. DistilBERT’s performance reflects both the transformer self-attention architecture and the scale of its pre-training corpus, and these two factors cannot be disentangled within this experimental setup. A transformer model trained exclusively on the 4,000 task examples would not be expected to perform comparably to fine-tuned DistilBERT, and any performance gap between DistilBERT and the other models therefore reflects the combined effect of architectural design and pre-training data scale rather than architecture alone. This distinction is important for interpreting the results in Section 5.

4 Implementation

4.1 Dataset

All four models are trained and evaluated on the IMDb movie review dataset, a standard benchmark for binary sentiment classification consisting of 50,000 reviews with balanced classes (7). Due to the CPU-only computing environment, training was limited to 4,000 examples, with 500 examples reserved for

validation and 500 held out as the test set. Both evaluation splits were drawn from the original IMDb test partition to ensure no overlap with the training data. Class balance is approximately preserved across all three splits, and a fixed random seed of 42 was applied throughout to ensure reproducibility across data splits, weight initialisations, and batch ordering.

4.2 Preprocessing

All reviews undergo a shared preprocessing pipeline: HTML tags are removed, text is lowercased, and leading and trailing whitespace is stripped. No stopword removal or stemming is applied, as both operations would interact differently with each model and introduce confounds into the comparison. Stopword removal in particular would reduce the vocabulary available to the LSTM and Word2Vec models without providing a commensurate benefit, since these models can learn to downweight uninformative terms through training. The TF-IDF vectoriser applies its own internal tokenisation, while the Word2Vec and LSTM models use whitespace-based splitting, a minor inconsistency acknowledged as a limitation of the current implementation.

4.3 Model Configuration

Key hyperparameters for all four models are summarised in Table 1.

Table 1: Hyperparameter configuration for all four models.

Model	Vocab/Emb	Hidden	Opt.	LR	Batch
Logistic Regression	10,000 / -	-	LBFGS	-	-
Word2Vec MLP	- / 50	128	Adam	1×10^{-3}	-
LSTM	20,000 / 100	128	Adam	1×10^{-3}	32
DistilBERT	- / 768	-	Adam	2×10^{-5}	16

The TF-IDF vocabulary was capped at 10,000 features with bigrams included alongside unigrams. Sublinear TF scaling was applied and L2 regularisation was set to $C = 1.0$. LBFGS was selected as the optimiser given its suitability for convex objectives and its reliable convergence without learning rate tuning. For the Word2Vec MLP, embeddings of dimension 50 were trained on the training reviews using the skip-gram objective with a context window of five. The MLP consists of a single hidden layer of size 128 with ReLU activation. The LSTM uses a vocabulary of 20,000 tokens with sequences truncated or padded to 200 tokens, balancing coverage of review length against computational cost. The embedding dimension of 100 and hidden size of 128 were

chosen to provide sufficient capacity without overfitting on the limited training data. For DistilBERT, the pretrained distilbert-base-uncased encoder was fine-tuned with a linear classification head. Input sequences were truncated to 128 tokens to reduce CPU inference time, and a learning rate of 2×10^{-5} was used to avoid destabilising the pre-trained representations through large gradient updates.

4.4 Training Procedure

All models were trained on the same 4,000 examples and evaluated after each epoch on the 500-example validation set. Early stopping was applied to all models based on validation accuracy with a patience of three epochs, and the best model state was restored before final evaluation on the held-out test set. All random seeds were fixed to 42 across Python, NumPy, and PyTorch. Four metrics are reported on the held-out test set: accuracy, F1 score, AUC, and training time in seconds. Accuracy measures the proportion of correctly classified examples. F1 provides a balanced measure of classification quality accounting for both precision and recall. AUC evaluates ranking quality independently of the decision threshold. Training time is treated as a primary evaluation criterion given the computational constraints of this experiment. Full metric definitions are provided in Section 5.

5 Experimental Results, Analysis and Conclusion

5.1 Evaluation Metrics

Four metrics are reported on the held-out test set of 500 examples. Accuracy measures the proportion of correctly classified examples. The F1 score is the harmonic mean of precision and recall

$$F_1 = 2 * \frac{precision * recall}{precision + recall} \quad (7)$$

and penalises classifiers that achieve high precision at the cost of recall or vice versa, making it a more balanced indicator than accuracy when error distributions are asymmetric across classes. The area under the receiver operating characteristic curve (AUC) measures the probability that the model assigns a higher score to a randomly chosen positive example than a randomly chosen negative one, evaluating ranking quality independently of the decision threshold. Training time in seconds is treated as a primary evaluation criterion rather than a secondary obser-

vation, directly relevant to the central question of whether greater architectural complexity is justified under the constraints of this experiment.

Table 2: Test-set results for all four models.

Model	Accuracy	F1	AUC	Time (s)
Logistic Regression	0.8640	0.8528	0.9457	1.17
Word2Vec MLP	0.7460	0.7233	0.8269	32.22
LSTM	0.7000	0.7012	0.7693	123.25
DistilBERT	0.8320	0.8257	0.9307	3234.43

Table 2 presents the test-set results for all four models. The results do not follow the expected pattern in which architectural complexity corresponds to predictive performance. Logistic regression achieves the highest accuracy (0.8640), F1 (0.8528), and AUC (0.9457) of all four models. The LSTM achieves the lowest accuracy (0.7000) despite being the only model among the first three that explicitly represents word order. DistilBERT performs second overall but does not surpass logistic regression on any reported metric, at a computational cost over 2,700 times greater.

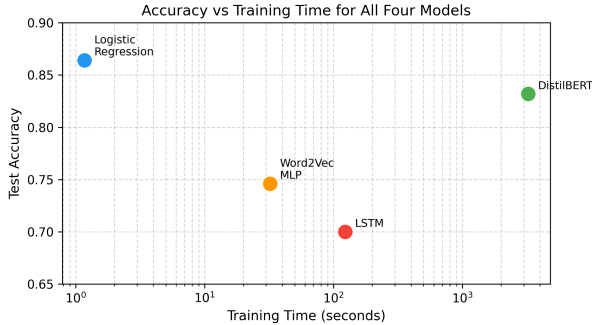


Figure 4: Test accuracy against training time for all four models. Logistic regression achieves the highest accuracy at negligible computational cost, while DistilBERT incurs over three orders of magnitude greater training time for lower accuracy on every reported metric. The horizontal axis is plotted on a log scale.

5.2 Results

RESULTS SUMMARY				
Model	Accuracy	F1	AUC	Time
Logistic Regression	0.8640	0.8528	0.9457	1.17s
Word2Vec MLP	0.7460	0.7233	0.8269	32.22s
LSTM	0.7000	0.7012	0.7693	123.25s
DistilBERT	0.8320	0.8257	0.9307	3234.43s

Figure 5: Test-set results for all four models on the 500-example held-out test set. Accuracy, F1, and AUC are reported alongside training time in seconds. Higher is better for all three predictive metrics. Training time is included as a primary evaluation criterion given the CPU-only experimental constraints.

The test-set results for all four models are presented in Table 2. The results do not follow the expected pattern in which architectural complexity corresponds to predictive performance. Logistic regression achieves the highest accuracy, F1, and AUC of all four models, while the LSTM achieves the lowest accuracy despite being the most expressively sequential of the first three models. DistilBERT performs second overall but does not surpass the logistic regression baseline on any metric.

5.3 Analysis

5.3.1 Logistic Regression

The strong performance of TF-IDF logistic regression reflects a close match between the model’s representational assumptions and the structure of the task. Sentiment classification on movie reviews is driven primarily by highly diagnostic lexical cues: terms such as "compelling," "tedious," and "incoherent" carry strong and consistent sentiment signal regardless of their sequential position. The implemented TF-IDF pipeline assigns high weights to terms that are frequent in a given review but rare across the corpus, directly capturing this signal. The inclusion of bigrams in the vocabulary allows two-word phrases such as "not good" to be treated as single features, partially addressing negation without requiring any sequential modelling. The convex optimisation objective with L2 regularisation and LBFGS guarantees convergence to the global minimum without the instability that gradient-based methods can exhibit on high-dimensional sparse inputs. The AUC of 0.9457 is the highest of all four models, indicating that logistic regression produces not only the most accurate hard predictions but also the best-calibrated probability estimates across all decision thresholds. The confusion matrix in Figure 2a shows 235 true negatives and 197 true positives against 24 false positives and 44 false negatives, a balanced error distribution with only a slight tendency to under-predict positive sentiment.

5.3.2 Word2Vec MLP

The Word2Vec MLP achieves an accuracy of 0.7460 and AUC of 0.8269, substantially below logistic regression despite using richer word representations. Two implementation-level limitations compound here. First, the Word2Vec embeddings were trained from scratch exclusively on the 4,000 training reviews. The skip-gram objective estimates co-occurrence statistics, and a corpus this small cannot reliably characterise rare or domain-specific sen-

timent vocabulary, producing embedding geometry that is noisy and incomplete. Second, the document representation is constructed by mean pooling all token embeddings, an operation that discards word order entirely and treats "the film was not good" identically to "the film was good." The model therefore inherits the bag-of-words limitation of TF-IDF while also paying the cost of learning representations from insufficient data, rather than benefiting from TF-IDF's principled frequency weighting which requires no learning at all. The confusion matrix in Figure 2b shows 52 false positives and 75 false negatives, an asymmetry indicating the model more frequently misses positive reviews than it incorrectly labels negative ones, consistent with embeddings that under-represent positive sentiment vocabulary.

5.3.3 LSTM

The LSTM achieves the lowest accuracy of all four models at 0.7000 and an AUC of 0.7693, the most counterintuitive result in the comparison. The explanation lies in the demands the implemented pipeline places on the available training data. The LSTM's embedding layer was randomly initialised and trained jointly with the recurrent weights and gating matrices from the same 4,000 examples. This means the model must simultaneously resolve representation learning and sequential reasoning from a single limited signal, with no prior knowledge of word meaning at the start of training. This contrasts directly with the Word2Vec MLP, where embeddings were at least exposed to the full training corpus through the skip-gram objective before classification training began. The LSTM therefore starts from a strictly weaker initialisation while facing a strictly harder optimisation problem. For reviews approaching the 200-token truncation limit, the hidden state at the final position must also compress and carry information from the full sequence, and early sentiment signal may be attenuated before reaching the classification decision. The confusion matrix in Figure 2c shows 85 false positives and 65 false negatives, a reversed asymmetry relative to the Word2Vec MLP, suggesting the model is biased toward predicting positive sentiment, consistent with representations that have not reliably separated the two classes during training.

5.3.4 DistilBERT

Fine-tuned DistilBERT achieves an accuracy of 0.8320 and AUC of 0.9307, the second-highest results overall. Its advantage over the Word2Vec and LSTM models is directly attributable to the pre-trained encoder, which enters fine-tuning with contextual repre-

sentations derived from approximately 16GB of text through masked language modelling. Fine-tuning at a learning rate of $2 * 10^{-5}$ adjusts these representations toward the task without overwriting them, meaning DistilBERT effectively has access to linguistic information that no amount of task-specific training on 4,000 examples could replicate for the other models. The self-attention mechanism allows every token to attend directly to every other token, enabling contextual representations that neither bag-of-words nor sequential models can produce. However, the fact that DistilBERT still does not exceed logistic regression on any metric suggests that the additional contextual modelling capacity it provides is not required for a task where the discriminative signal is primarily lexical. The confusion matrix in Figure 2d shows a perfectly symmetric error distribution of 42 false positives and 42 false negatives, indicating that the pre-trained representations provide equally reliable signal across both sentiment classes. Despite this, the training time of 3234.43 seconds against 1.17 seconds for logistic regression represents over three orders of magnitude difference for a model that achieves lower accuracy, F1, and AUC on every reported metric.

5.4 Confusion matrix Analysis

The four confusion matrices reveal systematic differences in error behaviour across models that scalar metrics alone do not capture. Logistic regression produces the most balanced error distribution, with false positives and false negatives differing by a factor of approximately 1.8, consistent with its high F1 and AUC. DistilBERT achieves perfect error symmetry across classes, reflecting the class-neutral quality of its pre-trained representations. The Word2Vec MLP and LSTM both show class-level asymmetry but in opposite directions: the Word2Vec MLP under-predicts positive sentiment while the LSTM over-predicts it, suggesting that their respective failure modes arise from different properties of their learned representations rather than a shared deficiency. This divergence is informative because both models fail for data-related reasons, but the nature of that failure differs depending on whether the representational bottleneck is embedding quality or joint optimisation under random initialisation.

5.5 Conclusion

The central finding of this experiment is that increased model complexity does not reliably produce better predictive performance under conditions of

limited training data and CPU-only compute. Logistic regression, trained in under two seconds on sparse TF-IDF features, outperforms all three neural models on every reported metric. This outcome is not a general indictment of neural approaches but a consequence of a specific combination of task characteristics and experimental constraints: sentiment classification is a lexically driven task, and TF-IDF logistic regression is particularly well-matched to that signal, while the neural models require data and compute conditions that this experiment does not provide. The three orders of magnitude difference in training time between logistic regression and DistilBERT, with no corresponding performance gain, makes the cost-benefit case for architectural complexity in this setting demonstrably weak. These findings suggest that the value of representational sophistication is strongly moderated by task structure, data scale, and available compute, and that principled simple baselines should be rigorously evaluated before more complex architectures are adopted.

References

- [1] B. Pang, L. Lee, and S. Vaithyanathan, “Thumbs up? sentiment classification using machine learning techniques,” in *Proceedings of the 2002 conference on empirical methods in natural language processing (EMNLP 2002)*, pp. 79–86, 2002.
- [2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- [6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [7] A. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, pp. 142–150, 2011.

6 Appendix

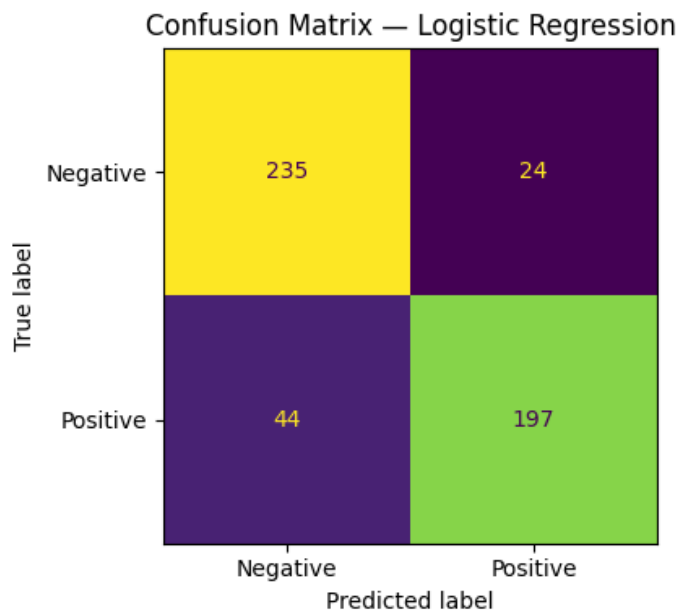


Figure 6: Confusion matrix for the TF-IDF + Logistic Regression model on the test set.

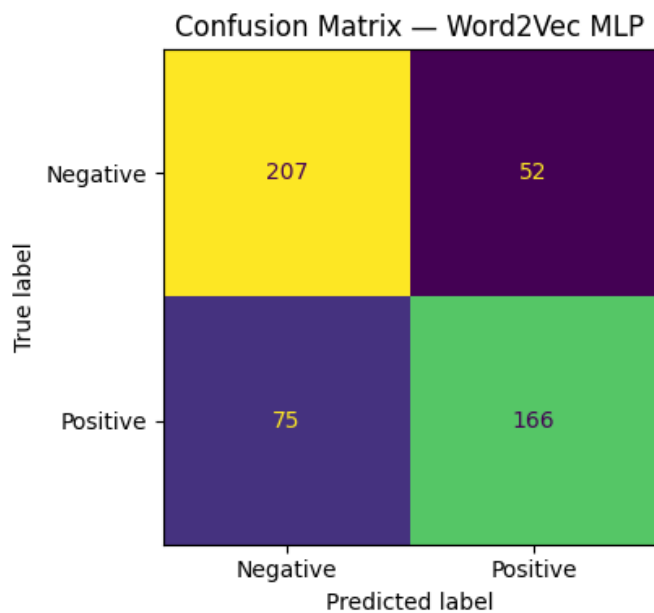


Figure 7: Confusion matrix for the Word2Vec + Mean Pooling + MLP model on the test set.

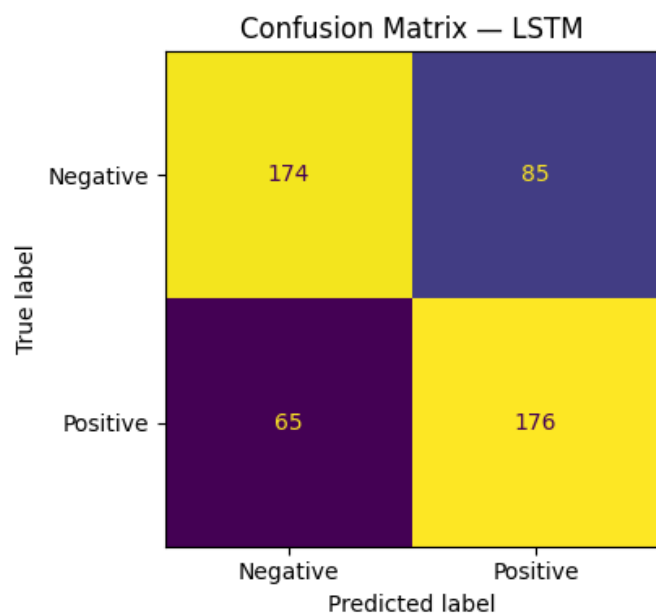


Figure 8: Confusion matrix for the LSTM model on the test set.

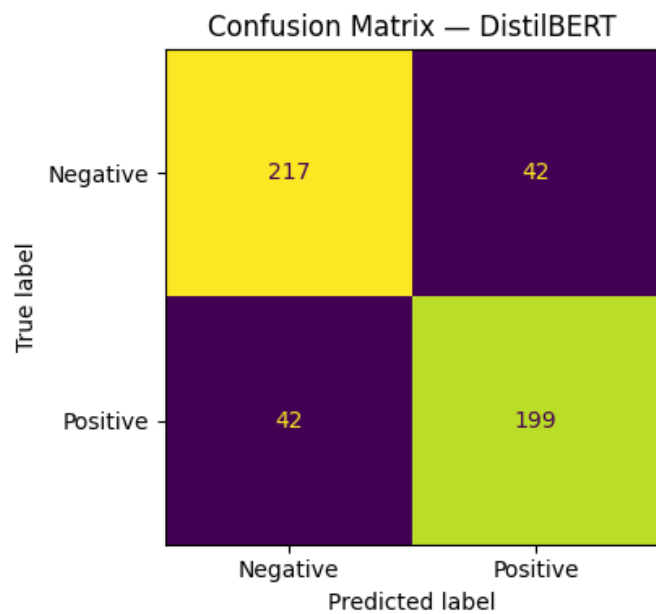


Figure 9: Confusion matrix for the fine-tuned DistilBERT model on the test set.